

02

SECTION 02

LLM 서비스 아키텍처 패턴

Gateway 하나로 가용성·비용·관측성 통합

AI SERVICE ARCHITECT / 2026 / 1H 30M

— LEARNING GOALS

오늘의 목표

학습 목표 5가지

- Gateway가 해결하는 7가지 문제 & 6가지 책임
- 멀티 프로바이더 모델 카탈로그 구분 (2026-04)
- Fallback / Routing / LB 전략 매칭
- Prompt Management = Code 원칙 + 대표 툴
- 3가지 표준 캐시 (Prompt · Exact · Semantic) 선택 근거

오늘의 데크 구조

- Opening · 표지 · 학습 목표
- Ch1 왜 Gateway인가?
- Ch2 Gateway 구축 옵션
- Ch3 라우팅 & LB
- Ch4 Fallback & 재시도
- Ch5 Prompt 관리
- Ch6 캐싱 (Prompt · Exact · Semantic)
- Closing · 통합 아키텍처 · 체크리스트

CHAPTER 1 / 6

01 / 6

왜 GATEWAY인가?

이 챕터에서 답할 질문

- 모델 하나만 호출하면 왜 안 되는가?
- 7가지 문제가 실제로 어떻게 사고로 이어지는가?
- Gateway 계층의 한 줄 원칙은 무엇인가?
- Gateway가 책임져야 할 7가지는 무엇인가? (오늘의 로드맵)

— WHY GATEWAY

"모델 하나만 쓰면 되지 않나요?"

서비스가 커질수록 동시에 닥치는 7가지 문제.

01	02	03	04	05	06	07
가용성	속도	비용	프롬프트 변경	관측성	거버넌스	다중 팀 공유

— INCIDENT · SEVEN FACES

7가지 문제의 실제 사고 사례

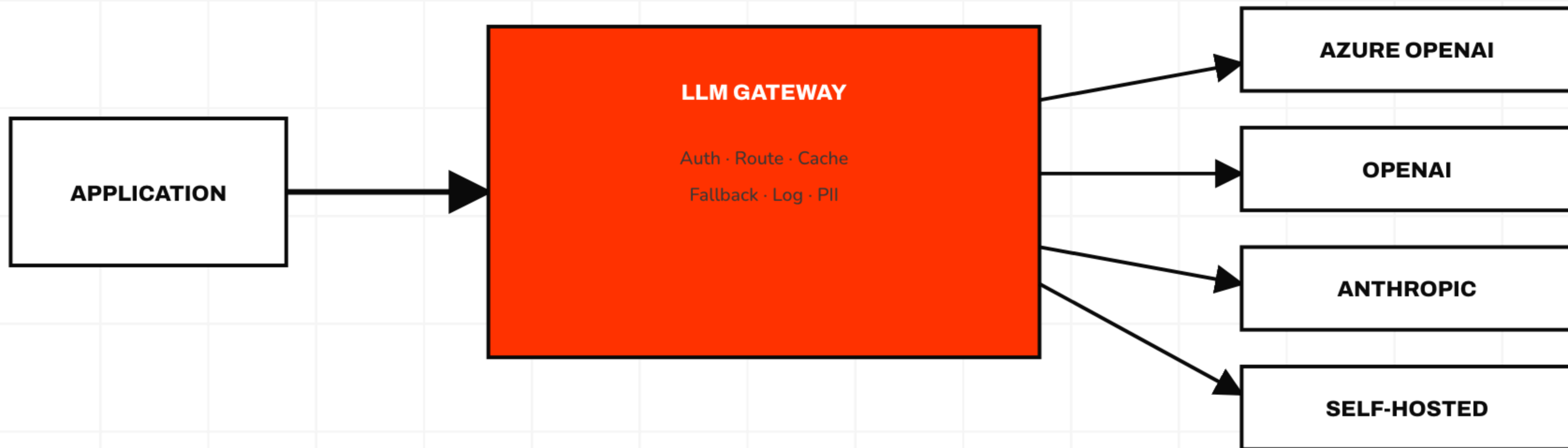
운영 현장에서 한 번은 만나는 장면들.

01 가용성 Azure East US 10분 장애 → 전 서비스 다운	02 속도 요약 API, 모델 교체 후 p95 300ms→9s	03 비용 사용자 10배 ↑, 청구서 \$2K→\$38K/월	04 프롬프트 한 줄 수정 → 응답 형식 붕괴, 롤백 불가	05 관측성 월말 청구서 와서야 특정 팀이 70% 썼다는 걸 알	06 거버넌스 프롬프트 로그에 주민번호 평문 노출, 감사 지적	07 다중 팀 A팀 이벤트성 트래픽이 B 팀 SLO를 깨뜨림
--	--	---	--	--	---	--

7가지 모두 실제 운영 사고. 앱 코드 분산 금지, Gateway 한 곳으로 집결.

— CORE PRINCIPLE

"앱은 **GATEWAY**에만 호출.
GATEWAY가 **PROVIDER**에 위임."



— RESPONSIBILITIES · 오늘 섹션의 로드맵

GATEWAY가 책임질 6가지

- 01 라우팅 & Load Balancing — 어느 모델·어느 배포본으로 보낼지 결정. Ch3
- 02 Fallback & 재시도 — 429·5xx → 대체 경로, 지수 백오프. Ch4
- 03 Prompt 관리 — Version · A/B · Eval · Observability. Ch5
- 04 캐싱 (3종 표준) — Prompt · Exact · Semantic. Ch6
- 05 관측성·비용 태깅 — 팀·서비스·기능 레벨 가시화. 횡단
- 06 안전·거버넌스 — PII · 금칙어 · 프롬프트 인젝션 · 감사 로그. 횡단

이 6 책임이 오늘 섹션의 축. 각 Chapter 표지에서 현재 위치를 재확인.

CHAPTER 2 / 6

02_{/6}

GATEWAY 구축 옵션

이 챕터에서 답할 질문

- 어떤 Gateway 플랫폼을 선택할 것인가? (4 카테고리)
- Cloud Native · SaaS · OSS Framework · 자체 구축은 뭐가 다른가?
- OSS Framework로 무엇이 가능한가? (LiteLLM 관리 콘솔)

— BUILD VS BUY · 4 PATHS

GATEWAY 구축 옵션 맵

01**CLOUD NATIVE**

Azure APIM + AI Gateway. 네이티브 통합 · 엔터프라이즈 표준.

02**SAAS**

Portkey · Helicone. 가장 빠른 도입 · 관측성 UI.

03**OPENSOURCE
FRAMEWORK**

LiteLLM. 완전 셀프호스팅 · 100+ Provider 통합.

04**자체 구축**

FastAPI 직접 구현. 제어권 100% · 유지비 100%.

자체 구축은 숨은 유지비 多. Cloud Native · SaaS · OSS Framework 우선 검토, 자체 구축은 최후 옵션.

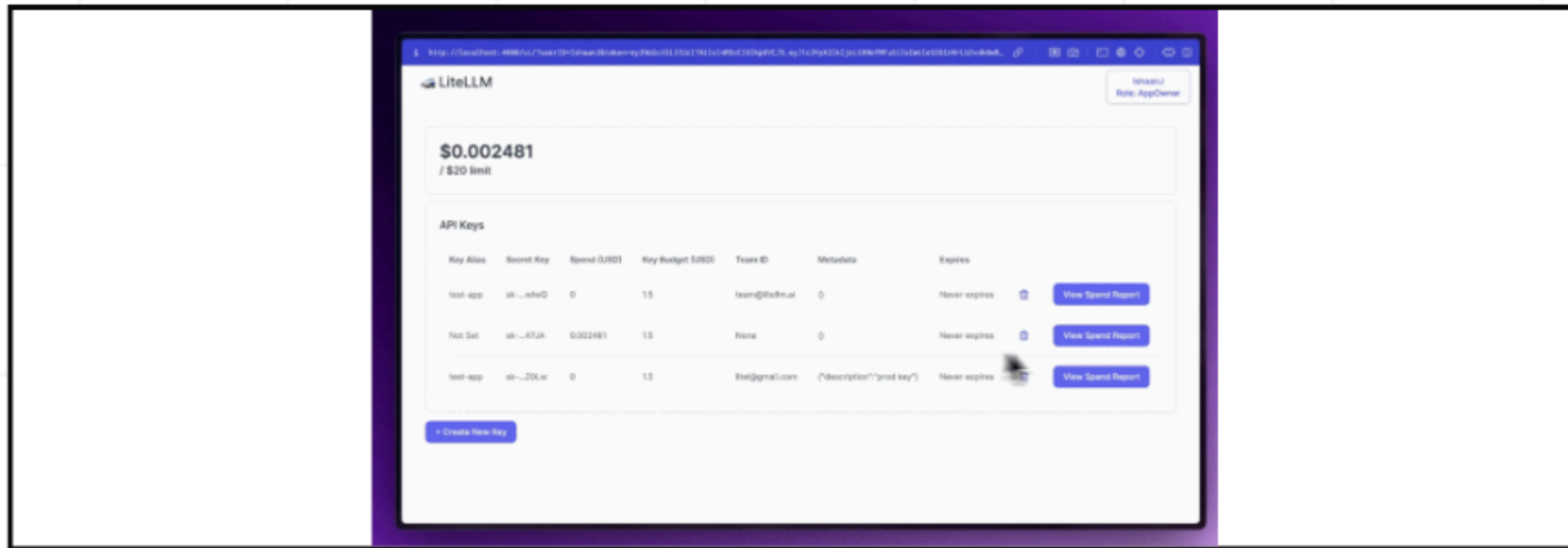
— 4-CATEGORY DEEP DIVE

CLOUD NATIVE · SAAS · OSS FRAMEWORK · 자체구축 **한판 정리**

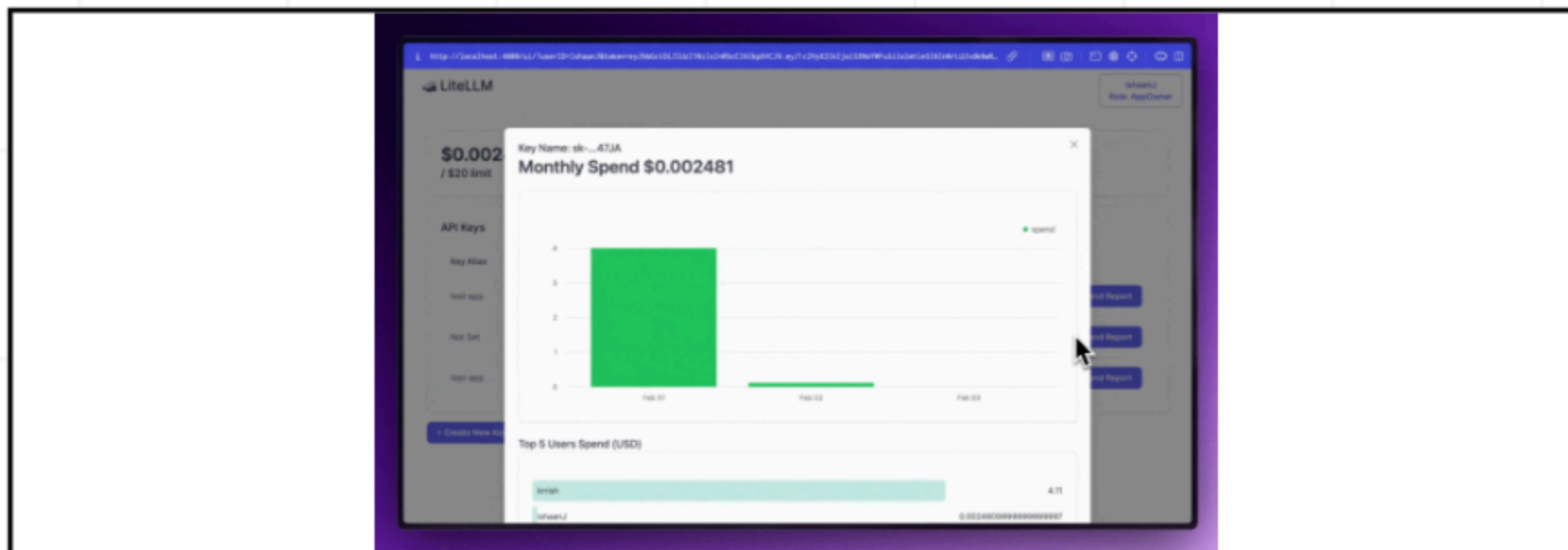
APIM Cloud Native 정체 Azure API Management + AI Gateway 정책 기능 Backend Pool · Retry · Circuit Breaker · Token Rate Limit · PII 강점 Entra ID·Key Vault·App Insights 네이티브 통합, Private Link 적합 Azure 중심 · 규제 산업	PORTKEY · HELICONE SaaS 정체 상용 LLM Gateway SaaS (Helicone은 OSS 버전 有) 기능 250+ Provider · Fallback · LB · Semantic Cache · Guardrail · Virtual Key · Cost Dashboard 강점 가장 빠른 도입, Helicone은 관측성·비용 분석 UI 최고 적합 멀티 프로바이더 · 빠른 프로덕션 · 비용 가시성 우선	LITELLM OSS Framework 정체 Python 라이브러리 + Proxy Server (MIT) 기능 100+ Provider · OpenAI 호환 API · Fallback · LB · Cache · Budget · Audit · Virtual Key 강점 오픈소스·무료, 완전 셀프호스팅, 데이터 유출 0 적합 비용 최소화 · SaaS 불가 · Python 스택	자체 구축 Custom 정체 FastAPI · Express 등 직접 구현 기능 요구사항대로 모두 직접 구현 (Routing · Cache · Rate Limit · 감사 로그) 강점 제어권 100%, 특수 라우팅·규제 요건 자유 적합 기성품으로 못 맞추는 규제·도메인 요건만
--	--	--	---

한 줄 의사결정 Azure 중심 → APIM · 빠른 멀티프로바이더 → Portkey·Helicone · 완전 셀프호스팅
·무료 → LiteLLM · 특수 요건만 → 자체 구축

LITELLM 관리 콘솔 — 가상 키 · 예산 · 사용량



팀·앱별 가상 키 + 키 단위 예산·만료



키별 일자 사용량 + 사용자 Top N 분해

→ LiteLLM Proxy Admin UI · docs.litellm.ai

핵심 기능 한눈에

- **Virtual Key** — 팀·서비스·앱 단위 가상 키 발급, 업스트림 키는 Vault에 격리
- **Key Budget** — 키별 USD 한도 + 자동 차단, 예산 초과 시 호출 거부
- **Team / Metadata** — 비용 태깅으로 "어느 팀·기능"을 분해
- **Spend Report** — 월간 추이 + 사용자 Top N 분해, 청구서 사후분석 0단계
- **Expiry** — 키 자동 만료, 퇴사·롤오버 자동화

왜 OSS FRAMEWORK가 강조되는가

- SaaS 동급 기능을 **완전 셀프호스팅**으로 확보 → 데이터 유출 0
- 월 라이선스 비용 0, OpenAI 호환 API 한 줄 교체

CHAPTER 3 / 6

03 / 6

라우팅 & **LOAD BALANCING**

이 챕터에서 답할 질문

- 3가지 라우팅 전략(Priority / Weighted / Intent)은 어떻게 다른가?
- 2026-04 기준, 어떤 모델이 어떤 업무에 맞는가?
- Cascade로 비용을 어떻게 5~10배 줄일 것인가?

— ROUTING STRATEGY

3가지 라우팅 전략

01

PRIORITY FALLBACK

장애 대응. 1 → 2 → 3순위. 429/5xx/timeout 트리거.

02

WEIGHTED LB

분산·쿼터 회피. 동일 모델 멀티 리전 50/50. 확률적 분배.

03

INTENT / SEMANTIC

품질·비용 균형. 쿼리 분류 후 모델 선택.

세 전략은 대체재가 아닌 **보완재**. 실전에서는 중첩 적용.

— DEEP DIVE · PER-STRATEGY

라우팅 전략 상세 원리

01 PRIORITY FALLBACK	동작 우선순위 순서대로 시도 → 실패 시 다음 타겟 즉시 전환 (백오프 X)	트리거 429 · 5xx · timeout · content- filter FP · CB open	구성 우선순위 리스트 + 트리거 조건 세트 (cross-target 전환 그 자체)	장/단 단순·예측 가능 / 다단계 실패 시 지 연 누적
02 WEIGHTED LB	동작 가중치대로 확률적 분배 (예: 80/20)	기준 정적 · 동적(지연/오류율) · 쿼터 기반	구성 가중치 + 분배 알고리즘 (random/RR/least-loaded) + sticky	장/단 비용·분산 동시 / 요청별 난이도 무시 · 예측 불가
03 INTENT / SEMANTIC	동작 분류기가 의도·난이도 판정 → 카테고 리별 모델 선택	분류 방식 규칙 · 소형 LLM · 임베딩 유사도 · 매니지드	구성 분류기 + 매핑 테이블 + 분류 실패 시 Fallback	장/단 요청별 정밀 최적화 / 분류 지연·비용 + 오분류 리스크

설계 포인트: 전략 단일 선택이 아닌 레이어별 전략 배치.

ROUTING · SCENARIOS

3 라우팅의 실전 시나리오

01

PRIORITY FALLBACK

실시간 고객 챗봇.

1순위 Azure East US

2순위 Azure West Europe

3순위 OpenAI 직접

트리거: 429 · 5xx · timeout>30s

02

WEIGHTED LB

동일 모델 멀티 리전 분산.

Azure East US 50%

Azure West Europe 50%

동일 모델 (예: GPT-5.4)

효과: 단일 리전 쿼터 회피 + 장애 격리

03

INTENT ROUTING

개발자 지원 Copilot.

"멀티스텝 코딩" → Opus 4.7

"간단 FAQ" → Haiku 4.5

"복잡 디버깅" → o4-mini

"초장문 문서" → Gemini 3.1 Pro

실무는 셋의 조합: Intent로 티어 결정 → 각 티어마다 Fallback → 같은 티어 안에서 Weighted LB.

멀티 프로바이더 모델 카탈로그

프로바이더	모델	I/O (\$/1M)	포지션·특징
OPENAI	GPT-5.5	신규	차세대 flagship · 2026-04 롤아웃
	GPT-5.4	2.50 / 15	안정 flagship · 프로덕션 표준
	GPT-5.4-mini	0.75 / 3	균형 티어 · 가성비
	GPT-5.4-nano	0.20 / 0.80	초경량 분류 · Cascade 1단
	o4-mini	1.10 / 4.40	Reasoning 전용 · o3-mini 후속
ANTHROPIC	Claude Opus 4.7	5 / 25	최신 flagship · 2026-04-16 · Coding·Agent 최강
	Claude Opus 4.6	5 / 25	안정 flagship · 프로덕션 검증
	Claude Sonnet 4.6	3 / 15	균형 · 128K 출력 + extended thinking
	Claude Haiku 4.5	1 / 5	저가·저지연 · Cascade 1단 유력
GOOGLE	Gemini 3.1 Pro	~3 / 15	Flagship · 2M 컨텍스트 · 멀티모달
	Gemini 3.1 Flash	~0.30 / 1.20	균형 · 빠른 범용 응답
	Gemini 3.1 Flash Live	스트리밍	실시간 음성·비전 · sub-1s TTFT
	Gemini 3.1 Flash Lite	~0.10 / 0.40	초경량 · 번역·분류 최저가

업무별 모델 선택 예시

업무	모델	이유
스팸·카테고리 분류	GPT-5.4-nano / Gemini 3.1 Flash Lite	저난도·고빈도 · 최저가
FAQ 자동 응답	Claude Haiku 4.5	짧고 반복됨, 저지연
고객 문의 요약	GPT-5.4-mini / Claude Sonnet 4.6	품질·비용 균형
긴 보고서 초안 생성	Claude Sonnet 4.6	128K 출력 + 추론
2M 토큰 규모 문서 분석	Gemini 3.1 Pro	2M 컨텍스트 유일급
멀티모달 영수증 OCR	GPT-5.4 / Gemini 3.1 Pro	네이티브 이미지 입력
실시간 음성 상담 봇	Gemini 3.1 Flash Live	sub-1s TTFT · 스트리밍
장기 Agent (멀티스텝 코딩)	Claude Opus 4.7	Coding·Agent 최강
복잡 수학·다단계 논리	o4-mini / Opus 4.7	Reasoning

체크 질문 "이 업무가 진짜 플래그십을 필요로 하는가?" 대부분의 답: "아니오".

— COST-OPTIMAL · 4-STAGE

CASCADE 라우팅 (2026-04)

1	HAIKU 4.5 / GPT-5.4-NANO 저가 티어 · \$1/\$5 또는 \$0.20/\$0.80 · 분류·FAQ·짧은 요약	70% 1단 종결
2	SONNET 4.6 / GPT-5.4-MINI 균형 티어 · \$3/\$15 또는 \$0.75/\$3 · 문의 요약·구조화 생성	20% 2단 종결
3	OPUS 4.7 / GPT-5.4 / GEMINI 3.1 PRO 플래그십 티어 · 긴 컨텍스트·코딩·복합 질의	7% 3단 종결
4	O4-MINI / OPUS 4.7 (EXTENDED) Reasoning 티어 · 수학·다단계 논리·검증 실패 재시도	3% 최종

효과 평균 토큰 비용 플래그십 단일 대비 5~10배↓ · 평균 지연 약 3배↓ (대부분이 저가·빠른 모델에서 종결)

CHAPTER 4 / 6

04_{/6}

FALLBACK & 재시도

이 챕터에서 답할 질문

- 언제 Fallback을 트리거할 것인가?
- 백오프 · Idempotency는 어떻게 보장하는가?
- Fallback 동작은 어떻게 관측할 것인가?

— FALLBACK DESIGN

FALLBACK 설계 체크

- 01 트리거 — 429 / 5xx / timeout / content-filter false-positive / CB open
- 02 백오프 — 같은 타겟 재시도에만 지수+지터. 다음 fallback 타겟 전환은 즉시
- 03 Idempotency — Tool 호출 재시도 시 치명적. 멍등 키 필수
- 04 관측성 — "왜 Fallback됐는가" trace에 기록

흔한 사고 멍등 키 부재 → 재시도 시 이메일 2회 발송, 결제 2회 청구.

FALLBACK 4요소 상세

01 트리거 (TRIGGER)	명확한 실패 429 rate limit · 5xx 서버 오류 · timeout(>N초)	숨은 실패 content-filter false positive · schema 검증 실패 · 응답 길이 0	상태 기반 Circuit Breaker open 상태면 해당 타겟 건너뛴	주의 4xx 중 400 Bad Request는 재시도 금지 — 요청 자체가 잘못된 경우 반복해도 실패	
02 백오프 (BACKOFF)	적용 범위 같은 타겟 재시도에만 · 다른 fallback 타겟 전환은 즉시 (지연 X)	지수 증가 1s → 2s → 4s → 8s (base × 2 ⁿ)	JITTER 각 지연에 ±20% 랜덤 추가 — thundering herd 방지	상한 max_delay = 60s · LLM 은 max_retries = 1~2회 권장	RETRY-AFTER 존중 서버가 Retry-After 헤더를 주면 계산값 대신 그 값을 사용
03 IDEMPOTENCY (멱등성)	왜 필요 재시도 시 Tool 호출 = 이메일 2번, 결제 2번 사고	키 설계 Idempotency-Key: {user}-{tool}-{hash(body)}-{ts}	저장소 Redis/DB에 24h TTL — 동일 키 재호출 시 캐시된 결과 반환	적용 범위 side-effect 있는 Tool 전부 — 결제·메일·DB write·외부 API	
04 관측성 (OBSERVABILITY)	필수 기록 Fallback 발생 시점·사유·대체 경로· 시도 횟수	TRACE 속성 fallback.reason · fallback.from · fallback.to · retry.count	알람 Fallback 비율 임계치 초과 시 알람 — Primary 장애 조기 감지	대시보드 경로별 성공률·평균 재시도 수 ·timeout 분포	

CHAPTER 5 / 6

05 / 6

PROMPT 관리

이 챕터에서 답할 질문

- 왜 "프롬프트는 코드"로 취급해야 하는가?
- 대표 툴(Langfuse / PromptLayer / Braintrust 등) 차이는?

— PROMPT MANAGEMENT

PROMPT = CODE

01

GIT 버전

변경 이력 추적

02

PR 리뷰

승인 워크플로우

03

배포 · 롤백

플래그 플립으로 즉시

04

자동 EVAL

regression set 실행

TOOL LANDSCAPE

대표 툴 비교

툴	특징	추천 상황
Langfuse (OSS)	Prompt 버전 + Trace + A/B	내부 호스팅 필요
PromptLayer	버전·Eval·Regression	SaaS로 바로 시작
Braintrust	Eval 중심 실험 플랫폼	평가가 최우선
Traceloop	Prompt Registry + OTel	OTel 기반 관측
Maxim AI	Prompt + Simulation + Eval	End-to-end 통합

CHAPTER 6 / 6

06 / 6

캐싱 (PROMPT · EXACT · SEMANTIC)

이 챕터에서 답할 질문

- LLM 서비스의 3가지 표준 캐시는 어떻게 다른가?
- Prompt / Exact / Semantic Cache 각각 무엇인가? (개념)
- Prompt Cache는 왜·어떻게 작동하나? (원리)
- Exact (Gateway KV) · Semantic (Vector)의 역할·리스크·적중률은?
- 어떤 순서로 도입할 것인가? 오적중 리스크는?

LLM 캐시 3가지 표준 유형

Portkey · LiteLLM · Langfuse · Redis · GPTCache 공통 용어.

01

PROMPT CACHE

Provider가 관리 · prefix 토큰 매칭
입력 50~90%↓ · 지연 최대 85%↓
리스크: 매우 낮음

02

EXACT CACHE

Gateway(Redis) · 전체 해시 일치
LLM 호출 0 · 지연 ~10ms · 비용 0
리스크: 낮음 (키 설계)

03

SEMANTIC CACHE

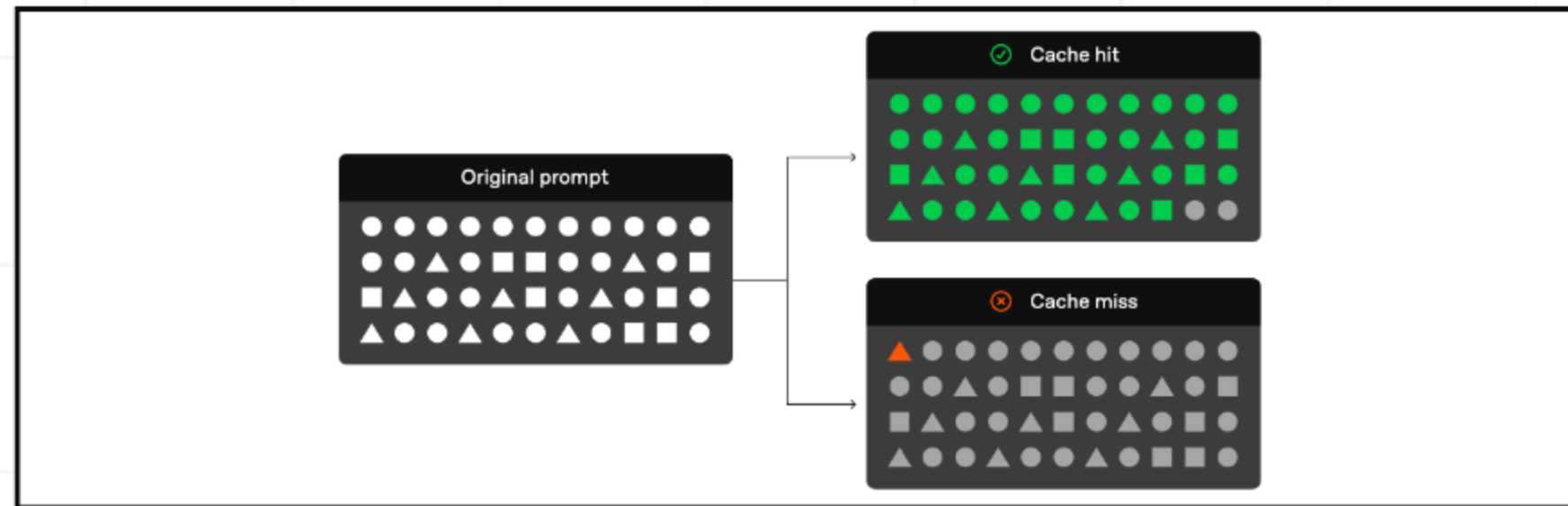
Gateway(VectorDB) · 의미 유사도
적중률 10~70% (도메인별) · +5~20ms
리스크: 높음 · 오적중 주의

분류 축 2개: (가) 관리 주체 (Provider vs Gateway) · (나) 매칭 단위 (prefix / 해시 / 의미).

— PROMPT CACHE · WHAT IT IS

PROMPT CACHE 한눈에 보기

같은 prefix면 HIT, 한 토큰만 달라도 MISS. prompt 설계가 곧 캐시 적중률.



토큰 시퀀스 단위 매칭



자연어 프롬프트 예시 - `Act` → HIT, `Behave` → MISS

→ Anthropic Docs · Prompt caching

핵심 메시지

- 같은 prefix → KV-state 재사용 → prefill 단계 건너뛰
- 한 글자라도 달라지면 처음부터 다시 prefill (뒤가 같아도 무효)
- prefix 일치 = 캐시 적중률의 전부. prompt 설계가 곧 비용

설계 함의 한 줄

- "불변 prefix 먼저, 가변 suffix 뒤" (다음 두 슬라이드에서 원리·구현)

PROMPT CACHE는 왜·어떻게 작동하는가

A. PREFILL VS DECODE 비용 구조

- Prefill — 입력 N 토큰 병렬 처리, $O(N^2)$, 전체 지연의 60~80%
- Decode — 출력 토큰 1개씩 순차 생성, 가벼운 루프
- Prompt Cache가 노리는 건 **Prefill 단계** — 여길 건너뛰면 비용·TTFT 동시 감소

B. CAUSAL ATTENTION 왜 prefix만?

- Token[n]의 KV-state = $f(\text{Token}[0..n])$
- 앞 토큰 하나만 바뀌어도 뒤 KV 전부 무효

C. BLOCK-LEVEL HASHING 어떻게 매칭?

- 토큰이 아니라 **블록(1,024~4,096 tokens)** 단위로 해시 비교
- 최장 일치 prefix 만큼만 KV 재사용, 뒤 블록부터 새로 prefill
- 최소 임계치(~1,024 토큰) 미만 프롬프트는 **캐시 대상 제외**

D. TTL과 GPU 메모리 왜 짧나?

- KV-state는 **GPU 메모리 상주** — 공간 유한
- LRU/TTL evict 필수 → OpenAI 자동 5~10분, Anthropic 5분/1시간
- Persistent TTL은 본질적으로 **GPU 자원 예약 비용**

설계 시사점 3가지: ① 맨 앞에 dynamic value 금지 · ② Few-shot·tool 정의 순서 고정 · ③ 짧은 프롬프트엔 Prompt Cache 대신 Exact Cache.

PROMPT CACHE **프로바이더 비교** (2026-04 기준)

PROVIDER	활성화	최소 PREFIX	가격 · TTL 핵심 (2026-04)
OpenAI	자동 (opt-out 불필요)	1,024 tokens	캐시 입력 $\times 0.5$ (-50%) · 자동 TTL 5~10분
Anthropic	명시적 <code>cache_control: {type:"ephemeral", ttl?: "1h"}</code> 최대 4 breakpoint / 요청	Opus 4.7/4.6/4.5: 4,096 Sonnet 4.6: 2,048 Haiku 4.5: 4,096	읽기 $\times 0.1$ (-90%) · 쓰기 $\times 1.25$ (5분) / $\times 2.0$ (1시간 GA) 워크스페이스 격리 ('26-02-05~) · 룩백 20 블록
Google (Gemini)	명시적 CachedContent API	4,096 tokens (Pro/Flash)	캐시 입력 -75% · TTL 임의 지정

설계 팁 [System Prompt] [Few-shot] [Tools] → 불변 (캐시 대상) · [사용자 질의] [Timestamp]
→ 가변 (캐시 제외).
흔한 실수: 시스템 프롬프트 맨 앞의 request-id·timestamp → 청구서 즉시 반토막.

EXACT CACHE는 무엇인가

전체 프롬프트 + 파라미터 + 모델의 해시로 매칭. 한 글자만 달라도 다른 키.

STORED

"비밀번호 어떻게 바꾸나요?"
hash = a1b2c3...

들어오는 쿼리

"비밀번호 어떻게 바꾸나요?"
"비밀번호 어떻게 바꾸나요?" (공백 1개 추가)
"비밀번호 어떻게 변경하나요?" (단어 차이)
"How to change my password?" (의미는 같음, 글자 다름)

HASH

a1b2c3
7f9d22
4e2c8a
b88e1f

결과

✓ Cache HIT
✗ Cache MISS
✗ Cache MISS
✗ Cache MISS

매칭 단위 = "전체 프롬프트 + 파라미터 + 모델"의 해시. 정규화(공백·줄바꿈) + 모델 버전 키 포함은 필수.

EXACT CACHE 키 설계 & 흐름

```
# 키 설계 · 4요소 필수
key = hash(
    model_id + model_version, # stale 방지
    normalized_prompt,       # 공백·줄바꿈 정규화
    temperature + top_p + seed, # 비결정성 파라미터
    tools_hash + max_tokens    # 응답 영향 요소
)
# 예: claude-opus-4-7-20260416-abc123...

# 호출 흐름
if cached := redis.get(key):
    return cached # ~10ms, LLM 0
resp = llm.call(...)
redis.set(key, resp, ttl=TTL)
return resp
```

TTL 가이드

- 정적 (공지·FAQ): 24h~7d
- 동적 (사용자 컨텍스트): 1h~6h
- 실험·디버깅: 5~15분
- 기본값: 1h

적합 시나리오

- FAQ · 배너 텍스트 · 공지 요약
- 반복 번역 · 테스트 환경
- 핫패스 트래픽 30~50%↓

③ SEMANTIC CACHE · WHAT IT IS

SEMANTIC CACHE는 무엇인가

임베딩 cosine 유사도 $\geq$ threshold면 HIT. 글자가 달라도 의미만 같으면 적중.

STORED

"비밀번호 어떻게 바꾸나요?"

→ *embedding* [0.12, -0.34, 0.55, ...]

THRESHOLD (예시)

cosine $\geq$ 0.90

들어오는 쿼리

	COSINE	결과
"비밀번호 어떻게 바꾸나요?"	1.000	✓ HIT (동일)
"비번 변경법 알려줘"	0.94	✓ HIT (의미 같음)
"패스워드 바꾸는 방법"	0.93	✓ HIT (의미 같음)
"How to change password?"	0.88	⚠ threshold 따라
"계정 어떻게 삭제하나요?"	0.62	✗ MISS

⚠ 위험: "계약 해지" vs "계약 체결" — 단어 중복으로 cosine 높음, 의미 정반대 (false positive 사고).

SEMANTIC CACHE 설계 & 리스크

파이프라인

- Query → Embedding → VectorDB Top-1 유사도
- threshold 이상 → **Cache 응답** (LLM 호출 0)
- miss → LLM 호출 + 새 벡터 저장
- 임베딩·유사도 오버헤드 5~20ms

도메인별 적중률

- FAQ·고객지원: 40~70%
- 번역·요약: 25~45%
- 문서 Q&A: 15~30%
- 오픈 채팅·창작: 5~20%

THRESHOLD 튜닝 (COSINE)

- 0.95~1.00: 매우 엄격 · 적중 낮음
- **0.90~0.95: 권장 시작점**
- 0.80~0.90: 느슨 · 오적중 주의
- <0.80: 위험 · 엉뚱한 답

오적중 방지 3종

- ① 안전 도메인부터 (결제·해지 제외)
- ② 사용자 피드백으로 오적중 모니터
- ③ 부정 키워드 필터 (해지·취소·삭제)

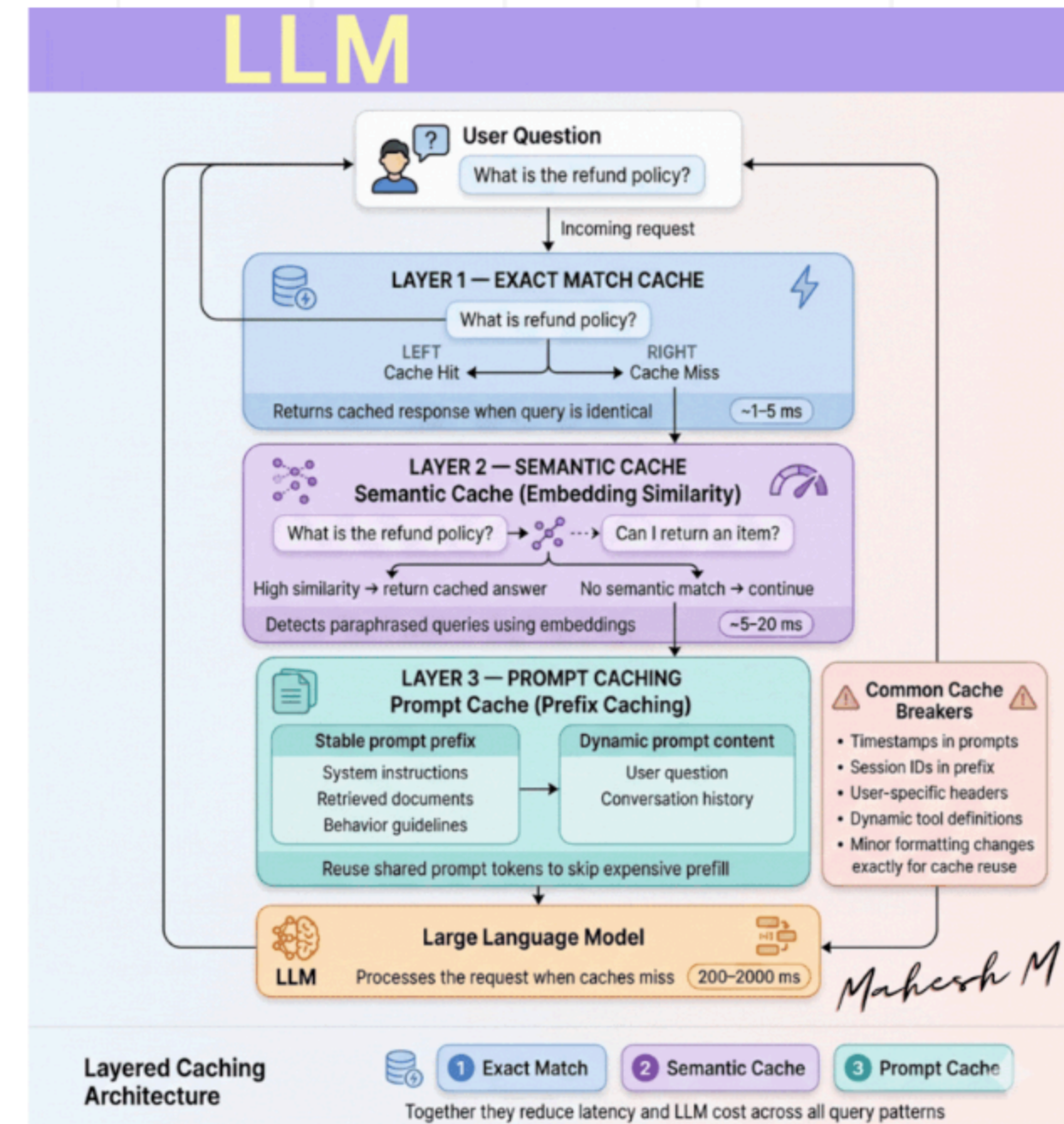
치명적 사고 예 "계약 해지는 어떻게?" → "계약 체결은 어떻게?" 답이 나가면 신뢰도 붕괴. 도입 전 반드시 안전장치 세 개 같이 올릴 것.

— ROLLOUT ORDER

캐시 적용 순서

도입 순서

- ① **Prompt Cache** — prefix 재구조화 (공짜) · 입력 50~90%↓ · 리스크 없음
- ② **Exact Cache** — Redis 핫패스 · 트래픽 30~50%↓ · 키 설계만 주의
- ③ **Semantic Cache** — Vector + 오적중 모니터 · FAQ 40~70%↓ · 도메인 의존



CLOSING

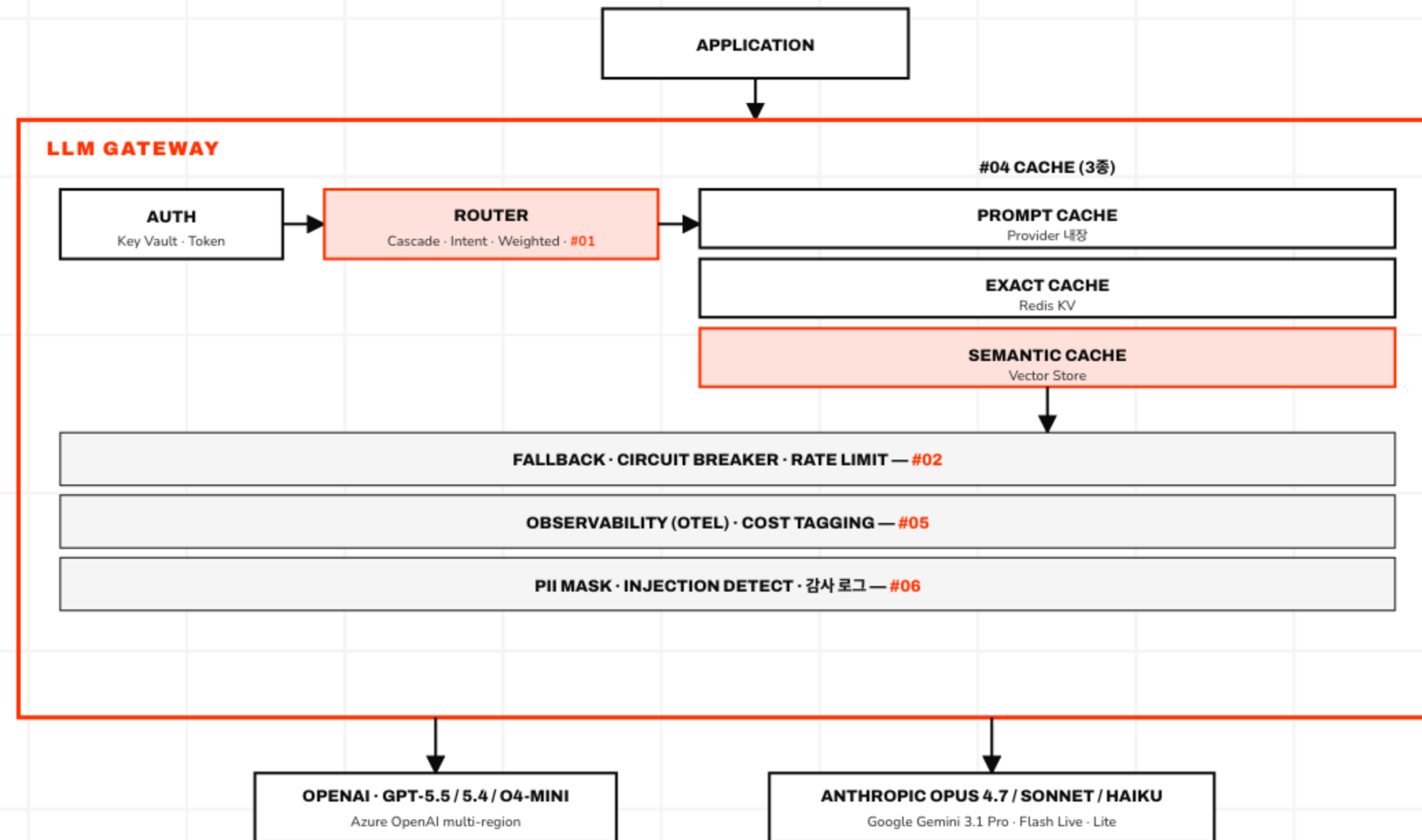
통합 & 마무리

이 마무리에서 하는 일

- 6 챕터 + 횡단 책임(관측성.거버넌스)을 한 장 아키텍처로 통합
- 오늘의 4 체크리스트로 학습 결과 자가 점검

— REFERENCE ARCHITECTURE

통합 참조 아키텍처



— SECTION 2 · WRAP-UP (4 책임 기준)

아키텍트 체크리스트

#01	라우팅 & LB CH3	3 전략 + 2026-04 카탈로그 + Cascade 4단 설계 가능
#02	Fallback & 재시도 CH4	트리거·백오프·Idempotency·관측성 통합 설계
#03	Prompt 관리 CH5	Prompt = Code 원칙 + 대표 툴(Langfuse 등) 차이 설명 가능
#04	캐싱 3종 CH6	Prompt·Exact·Semantic의 차이 + 도입 순서 Prompt→Exact→Semantic + 비용 레버 5가지

QUESTIONS ?